МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 2

з дисципліни "Професійний Python**"**

на тему "**Структури даних та функціональна декомпозиція**"

Виконав

студент 3 курсу

групи ФЕП-33

Рак Богдан Ігорович

Перевірив

Професор кафедри ОІТ

Свелеба Сергій Андрійович

Львів – 2026

Мета: Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю.

1. **Опис використаних структур даних**:

list

Використовується як основна впорядкована змінювана колекція для зберігання набору об'єктів Tech (весь список техніки, результати пошуку/фільтрації/сортування).

```
30  def create_demo_tech_list() -> list[Tech]:
31      return [
32          Tech(inventory_number="001", type="Laptop", vendor="Dell", model="XPS 13", price=999.99),
33          Tech(inventory_number="002", type="Smartphone", vendor="Apple", model="iPhone 13", price=799.99),
34          Tech(inventory_number="003", type="Tablet", vendor="Samsung", model="Galaxy Tab S7", price=649.99),
35          Tech(inventory_number="004", type="Laptop", vendor="HP", model="Spectre x360", price=1199.99),
36          Tech(inventory_number="005", type="Smartphone", vendor="Google", model="Pixel 6", price=599.99),
37          Tech(inventory_number="006", type="Monitor", vendor="Dell", model="UltraSharp U2720Q", price=549.99),
38          Tech(inventory_number="007", type="Laptop", vendor="Apple", model="MacBook Air M2", price=1099.00),
39          Tech(inventory_number="008", type="Printer", vendor="HP", model="LaserJet Pro", price=249.99),
40      ]
41
```

tuple

Використовується для незмінюваних довідкових даних - переліку допустимих типів техніки, а також для незмінюваного знімка історії пошукових запитів.

```
30  def create_demo_tech_list() -> list[Tech]:
31      return [
32          Tech(inventory_number="001", type="Laptop", vendor="Dell", model="XPS 13", price=999.99),
33          Tech(inventory_number="002", type="Smartphone", vendor="Apple", model="iPhone 13", price=799.99),
34          Tech(inventory_number="003", type="Tablet", vendor="Samsung", model="Galaxy Tab S7", price=649.99),
35          Tech(inventory_number="004", type="Laptop", vendor="HP", model="Spectre x360", price=1199.99),
36          Tech(inventory_number="005", type="Smartphone", vendor="Google", model="Pixel 6", price=599.99),
37          Tech(inventory_number="006", type="Monitor", vendor="Dell", model="UltraSharp U2720Q", price=549.99),
38          Tech(inventory_number="007", type="Laptop", vendor="Apple", model="MacBook Air M2", price=1099.00),
39          Tech(inventory_number="008", type="Printer", vendor="HP", model="LaserJet Pro", price=249.99),
40      ]
41
```

set

Використовується там, де потрібні лише унікальні значення без урахування порядку та кількості повторень - унікальні виробники та унікальні типи техніки.

```
60  def get_unique_vendors(tech_list: list[Tech]) -> set[str]:
61      return {tech.vendor for tech in tech_list}
62
```

dict

Використовується для побудови індексу «інвентарний номер -> Tech», що дозволяє переходити від лінійного пошуку O(n) до пошуку за ключем у середньому за O(1).

```
16  def create_tech_index(tech_list: list[Tech]) -> dict[str, Tech]:
17      return {tech.inventory_number: tech for tech in tech_list}
18
```

2. Використання модуля collections

Counter

Використовується для підрахунку кількості пристроїв кожного типу та кожного виробника.

```
82  def count_by_type(tech_list: list[Tech]) -> Counter:
83      return Counter(tech.type for tech in tech_list)
84
```

defaultdict

Використовується для групування пристроїв за типом, а також для вкладеного групування «виробник -> тип -> список пристроїв».

```
68  def group_by_type(tech_list: list[Tech]) -> dict[str, list[Tech]]:
69      grouped: defaultdict[str, list[Tech]] = defaultdict(list)
70      for tech in tech_list:
71          grouped[tech.type].append(tech)
72      return dict(grouped)
73
```

deque

Використовується для зберігання обмеженої (не більше 5 елементів) історії останніх пошукових запитів за моделлю.

```
 8  _search_history: deque[str] = deque(maxlen=5)
 9
10
11  def search_by_model(tech_list: list[Tech], model: str) -> list[Tech]:
12      _search_history.append(model)
13      return [tech for tech in tech_list if tech.model.lower() == model.lower()]
```

Comprehension & generator expression

List comprehension

```python
def filter_by_price_range(tech_list: list[Tech], min_price: float, max_price: float) -> list[Tech]:
    return [tech for tech in tech_list if min_price <= tech.price <= max_price]

```

Set comprehension

```python
def get_unique_types(tech_list: list[Tech]) -> set[str]:
    return {tech.type for tech in tech_list}
```

Dict comprehension

```python
def create_tech_index(tech_list: list[Tech]) -> dict[str, Tech]:
    return {tech.inventory_number: tech for tech in tech_list}
```

Generator expression

Генераторний вираз не створює проміжний список і використовується там, де потрібна лише одноразова ітерація для агрегації (сума, Counter).

```python
@measure_time
def avg_price(tech_list: list[Tech]) -> float:
    if not tech_list:
        return 0.0
    total_price = sum(tech.price for tech in tech_list)
    return total_price / len(tech_list)
```

```python
def count_by_type(tech_list: list[Tech]) -> Counter:
    return Counter(tech.type for tech in tech_list)

```

3. Функціональна декомпозиція

Логіка обробки даних розбита на невеликі функції, кожна з яких відповідає за одну задачу, і зібрана у файлі services.py:

Функція	Призначення
search_by_model	лінійний пошук за моделлю
create_tech_index	побудова dict-індексу за інвентарним номером
sort_by_vendor / sort_items	сортування за виробником / за довільним ключем
filter_by_price_range / filter_items	фільтрація за діапазоном ціни / за предикатом

create_price_filter / create_type_filter	closures - фабрики предикатів
get_unique_vendors / get_unique_types	унікальні значення (set comprehension)
group_by_type / group_by_vendor_and_type	просте та вкладене групування
count_by_type / count_by_vendor	статистика через Counter
avg_price / price_of_all_tech	агрегація: середня і сумарна вартість
find_most_expensive_tech / find_cheapest_tech	пошук екстремумів за ціною
average_of_values	приклад функції з *args
create_tech_record	приклад функції з **kwargs
process_pipeline	конвеєр послідовних перетворень списку
build_summary_report	формування підсумкового звіту (export)
benchmark_search	порівняння швидкодії пошуку в list і в dict

4. *args та **kwargs

```
113  def average_of_values(*prices: float) -> float:
114      if not prices:
115          return 0.0
116      return sum(prices) / len(prices)
```

```
119  def create_tech_record(**fields) -> dict:
120      return dict(fields)
```

5. Lambda та функції вищого порядку.

lambda використовується як компактний ключ сортування, а функції sort_items і filter_items є функціями вищого порядку - вони приймають іншу функцію (лямбду або звичайну функцію) як аргумент.

```python
elif choice == 4:
    sorted_tech = sort_items(tech_list, key=lambda tech: tech.price)
    print("\nSorted by Price:")
    print_tech_list(sorted_tech)
```

```python
96
97  def find_most_expensive_tech(tech_list: list[Tech]) -> Tech | None:
98      if not tech_list:
99          return None
100     return max(tech_list, key=lambda tech: tech.price)
```

6. Closure

Closure - функція, яка «пам'ятає» змінні зовнішньої функції навіть після завершення її виконання. У проєкті це фабрики предикатів для фільтрації:

```python
44  def create_price_filter(min_price: float) -> Callable[[Tech], bool]:
45
46      def predicate(tech: Tech) -> bool:
47          return tech.price >= min_price
48
49      return predicate
```

7. Decorator

Звичайний decorator measure_time вимірює час виконання функції:

```python
5   def measure_time(func):
6
7       @wraps(func)
8       def wrapper(*args, **kwargs):
9           start = perf_counter()
10          result = func(*args, **kwargs)
11          elapsed = perf_counter() - start
12          print(f"[measure_time] {func.__name__}: {elapsed:.8f} s")
13          return result
14
15      return wrapper
16
```

Параметризований decorator repeat(count) поєднує в собі decorator і closure, викликаючи функцію задану кількість разів:

```python
def repeat(count: int):

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(count):
                result = func(*args, **kwargs)
            return result

        return wrapper

    return decorator
```

8. Аналіз складності операцій

Операція	Складність	Приклад у проєкті
Лінійний пошук у list	O(n)	search_by_model
Побудова dict-індексу	O(n)	create_tech_index
Пошук за ключем у dict	O(1) у середньому	index.get(inventory_number)
Перевірка належності множині set	O(1) у середньому	vendor in unique_vendors
Сортування sorted()	O(n log n)	sort_by_vendor, sort_items

Big-O не показує точний час виконання, але дозволяє передбачити, як зростатиме час обробки зі збільшенням кількості даних: лінійний пошук сповільнюється пропорційно до n, тоді як пошук у dict/set залишається практично сталим.

9. Експерементальна частина

Для перевірки теоретичної складності на практиці було згенеровано набори даних із 1 000, 10 000 і 100 000 записів і виміряно час пошуку останнього

елемента (найгірший випадок для лінійного пошуку) через функцію benchmark_search.

Кількість records	list search, мс	dict search, мс
1 000	0.05032	0.00141
10 000	0.44537	0.00176
100 000	3.96322	0.00381

Висновок з експерименту: час лінійного пошуку в list зростає майже пропорційно до кількості записів (приблизно у 10 разів при збільшенні n у 10 разів), що відповідає теоретичній оцінці O(n). Час пошуку в dict залишається на порядки меншим і зростає значно повільніше, що відповідає очікуваній середній складності O(1). Це підтверджує, що для задач із частим пошуком за ключем побудова dict-індексу є виправданою навіть з урахуванням витрат O(n) на його побудову.

Висновок: Виконуючи лабораторну роботу, можна сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю.